iroh 전체 아키텍처 개요

소개

iroh는 공개키 기반의 P2P QUIC 연결을 제공하는 Rust 라이브러리입니다. 이 문서는 iroh의 핵심 아키텍처와 주요 컴포넌트들의 역할, 그리고 이들 간의 상호작용을 상세히 분석합니다.

iroh의 주요 목표는 두 노드 간에 직접적이고 암호화된 연결을 설정하는 것입니다. 이를 위해 hole-punching 기법과 relay 서버를 활용하여 NAT와 방화벽을 우회하고, 최종적으로는 직접 연결로 전환하여 최적의 성능을 제공합니다.

핵심 컴포넌트 분석

1. Endpoint - 연결 관리 및 API 진입점

역할과 책임:

- iroh 노드의 중앙 관리자 역할
- 다른 노드와의 연결 생성 및 수락
- QUIC 연결의 생명주기 관리
- ALPN(Application-Layer Protocol Negotiation) 프로토콜 협상

주요 API:

```
// 연결 생성
pub async fn connect(&self, node_addr: impl Into<NodeAddr>, alpn: &[u8]) -> R

// 연결 수락
```

```rust
pub async fn accept(&self) -> Option<Connecting>
```

```rust
// 노드 주소 추가
pub fn add_node_addr(&self, addr: NodeAddr) -> Result<(), AddNodeAddrError>
```

핵심 기능:

- **Builder 패턴**: 복잡한 설정을 단계적으로 구성
- **Discovery 통합**: 노드 발견 메커니즘과의 연동
- **Relay 모드 관리**: 기본 relay 서버 또는 커스텀 relay 설정
- **TLS 설정**: 공개키 기반 인증 및 암호화 구성

2. MagicSock - UDP 소켓 관리 및 패킷 라우팅

역할과 책임:

- UDP 소켓의 생성, 관리, 종료
- 패킷의 지능적 라우팅 (직접 연결 vs relay)
- 네트워크 경로 선택 및 최적화
- hole-punching 과정의 실행

주요 기능:

```rust
// 패킷 전송 준비
fn prepare_send(&self, udp_sender: &UdpSender, transmit: &quinn_udp::Transmit)
```

```rust
// 수신된 데이터그램 처리
```

```
fn process_datagrams(&self, bufs: &mut [io::IoSliceMut<'_>], metas: &mut [quinn
```

핵심 특징:

- **경로 다중화**: 동시에 여러 경로(직접 UDP, relay)를 통한 패킷 전송
- **자동 경로 전환**: 더 나은 경로 발견 시 자동 전환
- **네트워크 변화 감지**: 네트워크 상태 변화에 대한 실시간 대응
- **DISCO 프로토콜**: 노드 간 discovery 및 연결성 테스트

3. Discovery - 노드 발견 메커니즘

역할과 책임:

- 노드 ID를 기반으로 연결 정보 조회
- 노드의 주소 정보 게시 및 업데이트
- 다양한 discovery 방식의 통합 관리

지원하는 Discovery 방식:

DNS Discovery

```
// DNS 기반 노드 발견
pub struct DnsDiscovery {
resolver: DnsResolver,
pkarr_relay: PkarrRelay,
}
```

- DNS TXT 레코드를 통한 노드 정보 저장
- 표준 DNS 인프라 활용

- 글로벌 접근성 제공

mDNS Discovery (로컬 네트워크)

- 로컬 네트워크 내 노드 자동 발견
- Bonjour/Zeroconf 프로토콜 활용
- 설정 없이 즉시 사용 가능

Pkarr DHT Discovery

- Mainline DHT를 통한 분산 노드 발견
- 중앙 서버 없이 완전 분산 방식
- BitTorrent DHT 인프라 활용

Discovery 인터페이스:

```
pub trait Discovery: std::fmt::Debug + Send + Sync + 'static {
fn publish(&self, data: &NodeData) {}
fn resolve(&self, node_id: NodeId) -> Option<BoxStream<Result<DiscoveryItem, Di
fn subscribe(&self) -> Option<BoxStream<DiscoveryItem>>
}
```

4. Relay 시스템 - 연결 중계 및 fallback

역할과 책임:

- 직접 연결이 불가능한 경우 패킷 중계
- hole-punching 지원 서비스 제공
- 네트워크 상태 진단 도구 제공

Relay 프로토콜 특징:

- **암호화된 중계**: relay 서버는 패킷 내용을 볼 수 없음
- **HTTP 기반**: 표준 HTTP/HTTPS 프로토콜 사용
- **WebSocket 업그레이드**: 실시간 양방향 통신
- **QAD 지원**: QUIC Address Discovery 프로토콜 구현

Relay 연결 과정:

1. HTTP/HTTPS로 relay 서버 연결
2. WebSocket으로 프로토콜 업그레이드
3. 커스텀 relay 프로토콜로 전환
4. 암호화된 패킷 중계 시작

5. TLS/암호화 시스템 - 보안 및 인증

역할과 책임:

- Ed25519 공개키 기반 노드 인증
- QUIC TLS 연결의 암호화 설정
- 세션 키 관리 및 0-RTT 지원

핵심 특징:

```
pub struct TlsConfig {
secret_key: SecretKey,
cert_resolver: Arc<AlwaysResolvesCert>,
server_verifier: Arc<ServerCertificateVerifier>,
client_verifier: Arc<ClientCertificateVerifier>,
session_store: Arc<dyn ClientSessionStore>,
```

```
}
```

- **Raw Public Keys**: RFC 7250에 따른 공개키 직접 사용
- **상호 인증**: 클라이언트와 서버 모두 공개키로 인증
- **세션 재사용**: TLS 세션 티켓을 통한 0-RTT 연결
- **키 로깅**: 디버깅을 위한 TLS 키 로깅 지원

컴포넌트 간 의존성 관계

계층적 구조

```
┌─────────────────────────────────────┐
| Application |
├─────────────────────────────────────┤
| Endpoint |
├─────────────────────────────────────┤
| Discovery | TLS Config |
├──────────────────┼──────────────────┤
| MagicSock |
├─────────────────────────────────────┤
| UDP Transport | Relay Transport |
└─────────────────────────────────────┘
```

상호작용 패턴

1. 연결 설정 시:

- Endpoint → Discovery: 노드 주소 정보 조회
- Endpoint → MagicSock: 연결 요청 전달
- MagicSock → TLS: 암호화 설정 적용
- MagicSock → Transports: 실제 네트워크 연결

2. 패킷 전송 시:

- Application → Endpoint: QUIC 스트림 데이터
- Endpoint → MagicSock: UDP 전송 요청
- MagicSock → NodeMap: 최적 경로 선택
- MagicSock → Transports: 패킷 전송

3. 노드 발견 시:

- MagicSock → Discovery: 노드 정보 게시
- Discovery → External: DNS/DHT/mDNS 업데이트
- External → Discovery: 노드 정보 조회
- Discovery → MagicSock: 발견된 노드 정보 전달

데이터 흐름 분석

연결 설정 흐름

1. **주소 해결**: NodeId → NodeAddr (Discovery 활용)
2. **경로 설정**: 직접 주소 및 relay URL 준비
3. **QUIC 연결**: TLS 핸드셰이크 및 ALPN 협상
4. **경로 최적화**: hole-punching 시도 및 직접 연결 전환

패킷 라우팅 흐름

1. **목적지 분석**: NodeId 기반 매핑 주소 확인
2. **경로 선택**: 사용 가능한 모든 경로 평가
3. **동시 전송**: 직접 UDP 및 relay를 통한 병렬 전송
4. **경로 학습**: 응답 시간 기반 최적 경로 선택

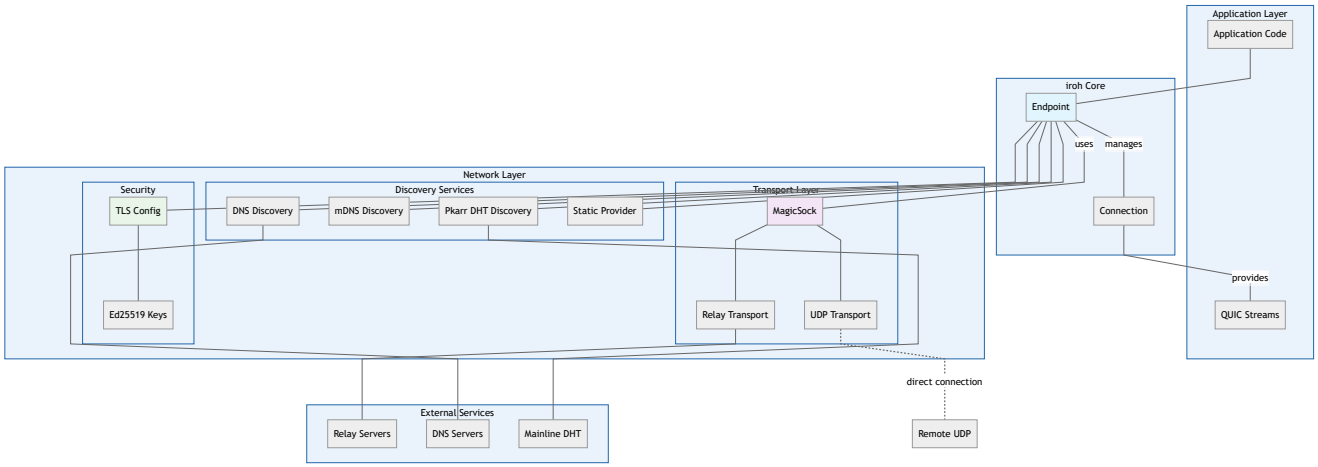
Discovery 정보 흐름

1. **정보 게시**: 로컬 주소 변경 시 discovery 서비스 업데이트
2. **정보 조회**: 연결 필요 시 타겟 노드 정보 조회
3. **캐시 관리**: 조회된 정보의 로컬 캐싱 및 만료 관리
4. **실시간 업데이트**: discovery 구독을 통한 실시간 정보 수신

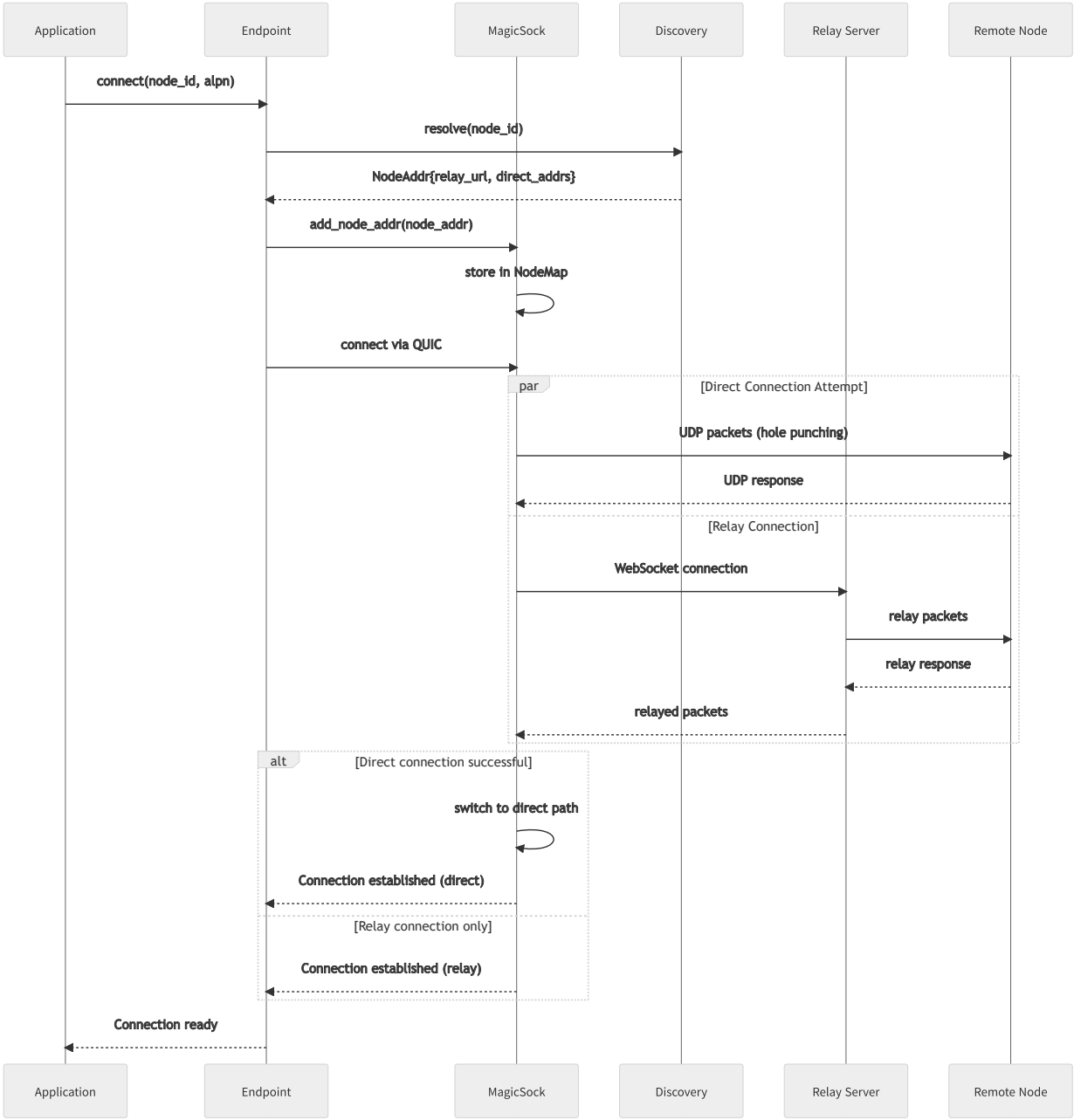
이러한 아키텍처를 통해 iroh는 복잡한 네트워크 환경에서도 안정적이고 효율적인 P2P 연결을 제공할 수 있습니다.

시스템 아키텍처 다이어그램

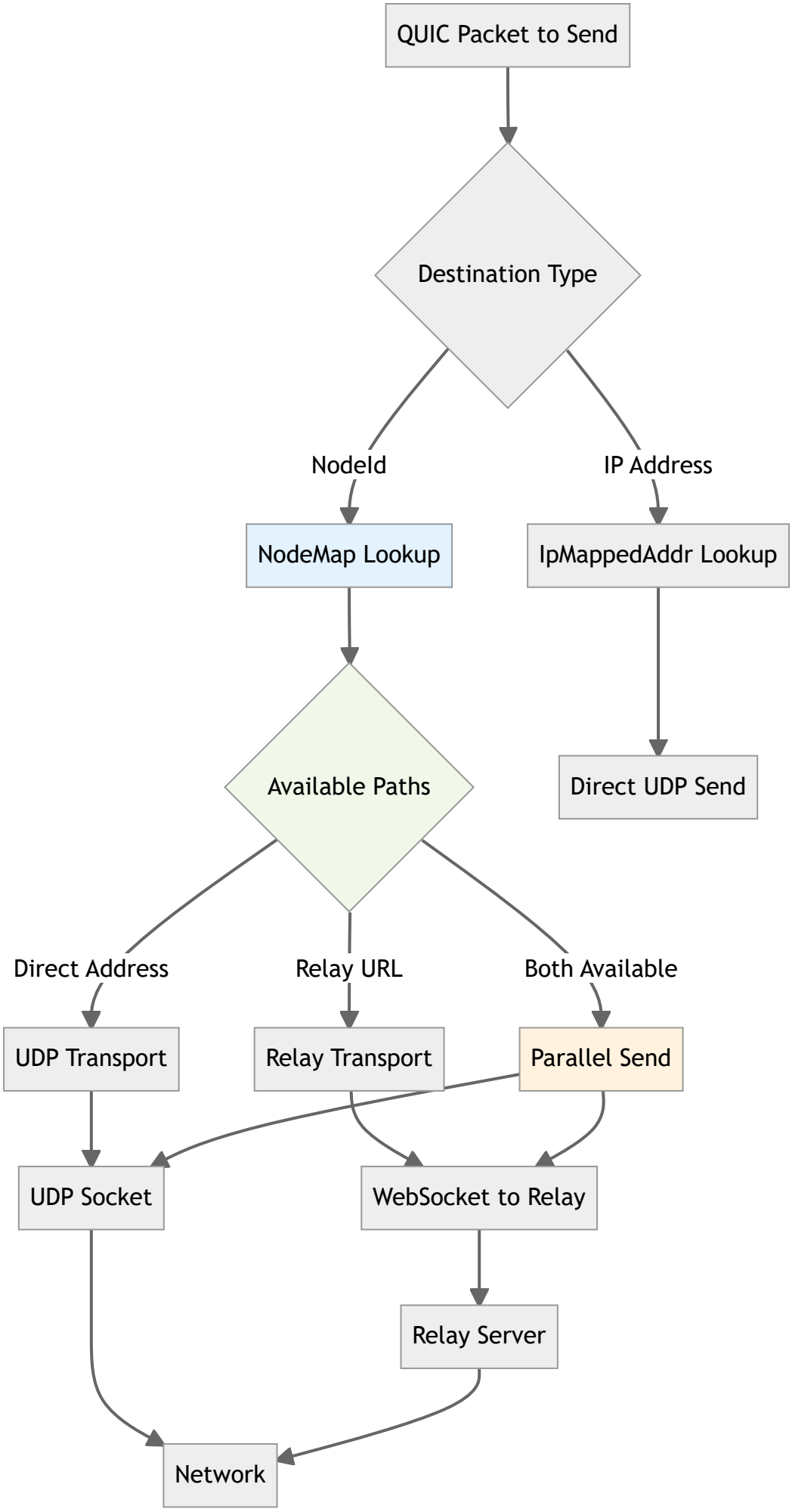
전체 시스템 아키텍처


Application Layer
Application Code
iroh Core
Endpoint
uses
manages
Connection
provides
QUIC Streams
Network Layer
Security
TLS Config
Ed25519 Keys
Discovery Services
DNS Discovery
mDNS Discovery
Pkarr DHT Discovery
Static Provider
Transport Layer
MagicSock
Relay Transport
UDP Transport
direct connection
Remote UDP
External Services
Relay Servers
DNS Servers
Mainline DHT

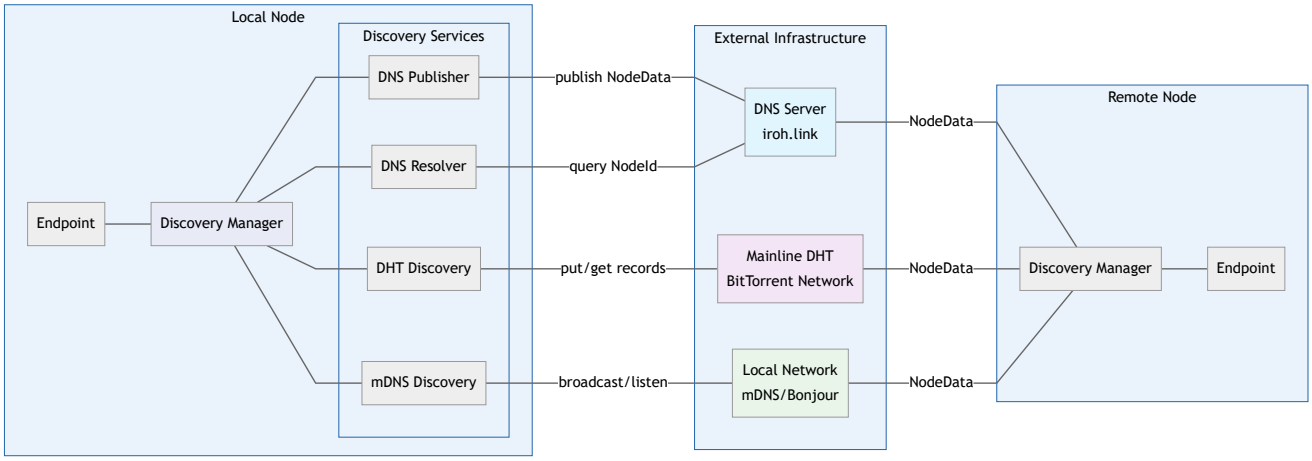
연결 설정 과정


Application
Endpoint
MagicSock
Discovery
Relay Server
Remote Node
connect(node_id, alpn)
resolve(node_id)
NodeAddr{relay_url, direct_addrs}
add_node_addr(node_addr)
store in NodeMap
connect via QUIC
par
[Direct Connection Attempt]
UDP packets (hole punching)
UDP response
[Relay Connection]
WebSocket connection
relay packets
relay response
relayed packets
alt
[Direct connection successful]
switch to direct path
Connection established (direct)
[Relay connection only]
Connection established (relay)
Connection ready
Application
Endpoint
MagicSock
Discovery
Relay Server
Remote Node

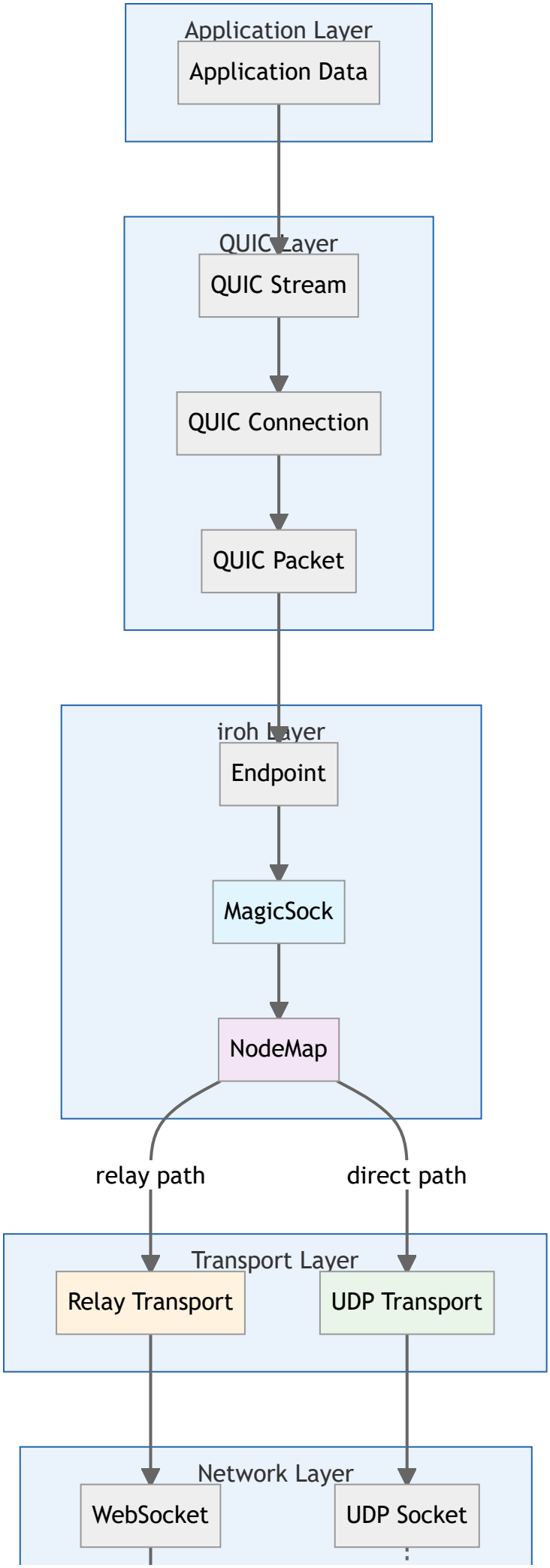
패킷 라우팅 흐름

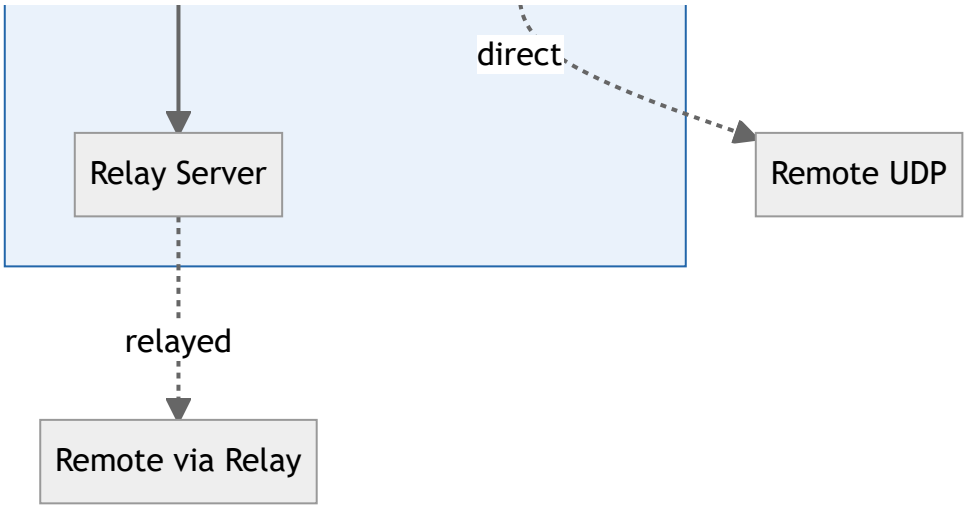

QUIC Packet to Send
Destination Type
NodeId
IP Address
NodeMap Lookup
IpMappedAddr Lookup
Available Paths
Direct UDP Send
Direct Address
Relay URL
Both Available
UDP Transport
Relay Transport
Parallel Send
UDP Socket
WebSocket to Relay
Relay Server
Network

Discovery 시스템 아키텍처

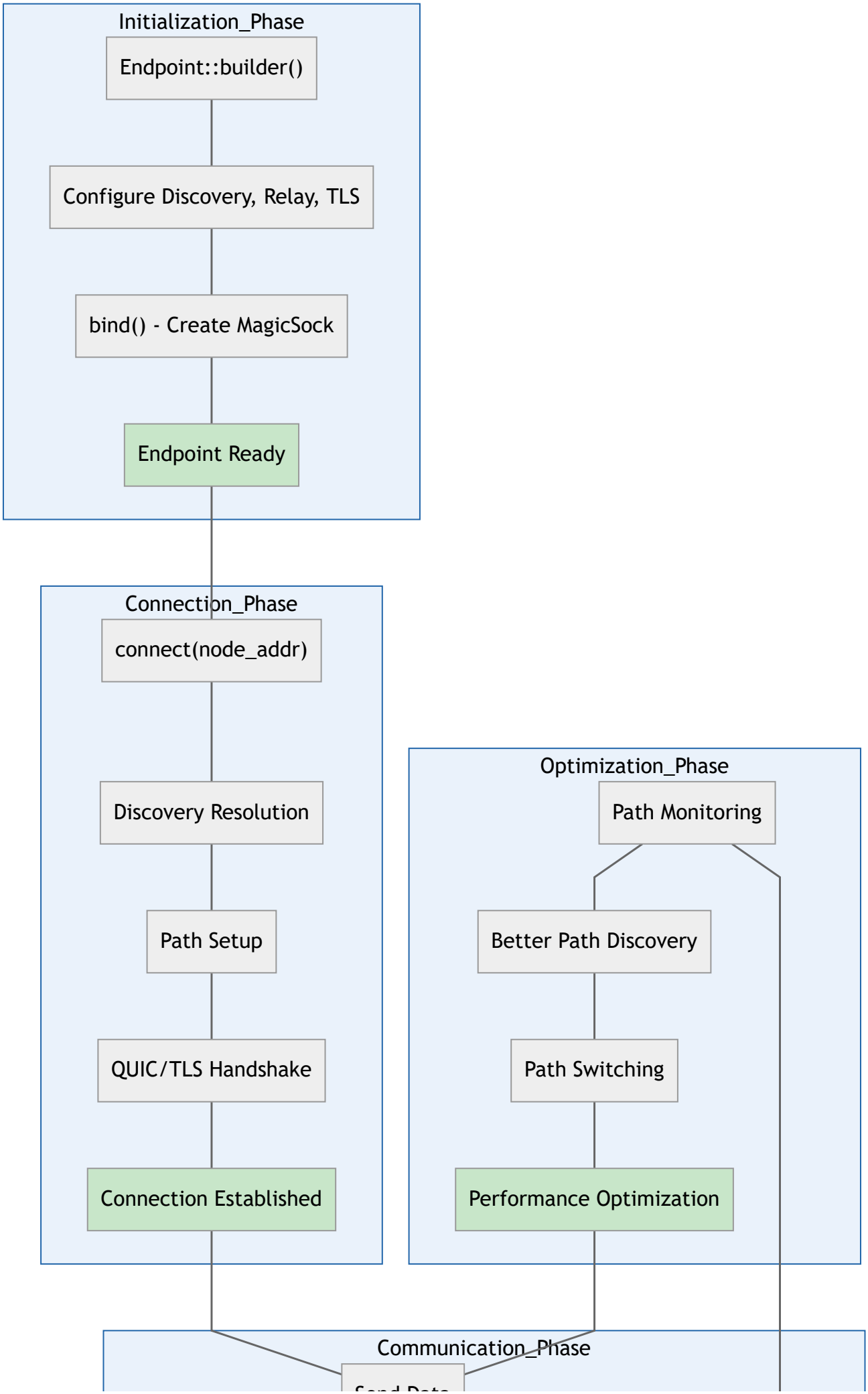


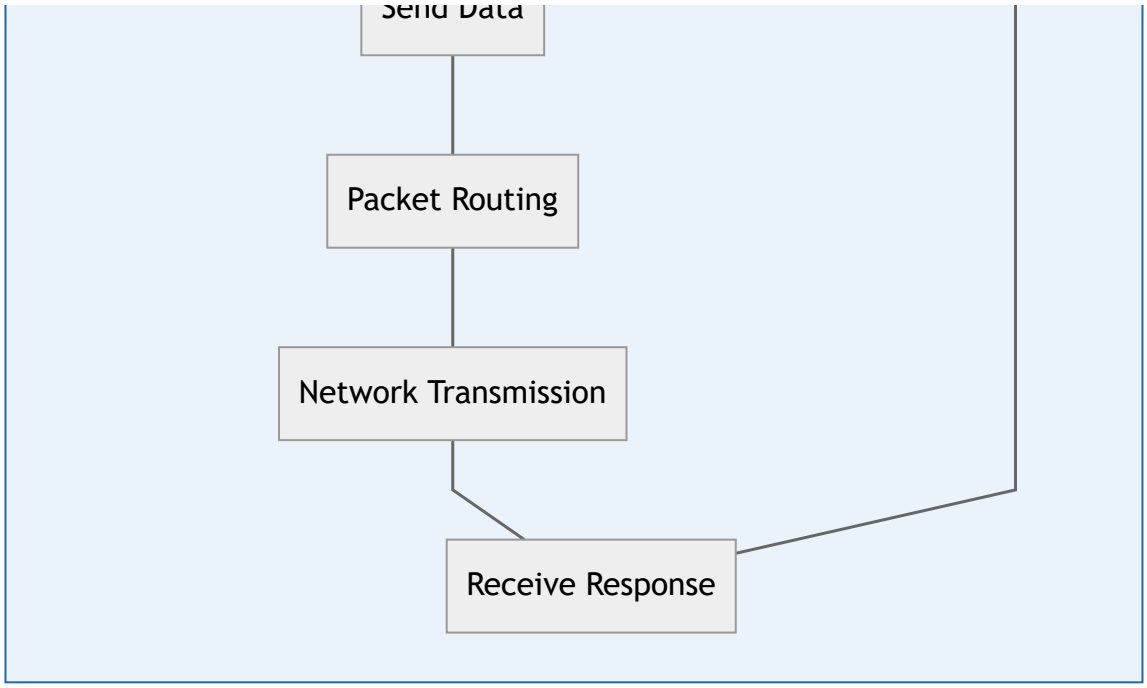
데이터 흐름 다이어그램


Application Layer
Application Data
QUIC Layer
QUIC Stream
QUIC Connection
QUIC Packet
iroh Layer
Endpoint
MagicSock
NodeMap
relay path
direct path
Transport Layer
Relay Transport
UDP Transport
Network Layer
WebSocket
UDP Socket



컴포넌트 상호작용 패턴


Initialization_Phase
Endpoint::builder()
Configure Discovery, Relay, TLS
bind() - Create MagicSock
Endpoint Ready
Connection_Phase
connect(node_addr)
Discovery Resolution
Path Setup
QUIC/TLS Handshake
Connection Established
Optimization_Phase
Path Monitoring
Better Path Discovery
Path Switching
Performance Optimization
Communication_Phase



이러한 다이어그램들은 iroh의 복잡한 아키텍처를 시각적으로 이해할 수 있도록 도와주며, 각 컴포넌트 간의 관계와 데이터 흐름을 명확히 보여줍니다.